Piston — Secure Code Execution Architecture & Pattern Extraction for Orchestration Workers

Security architecture review (evidence-based). Generated 2026-02-20.

Scope. Analyze Piston’s architecture for safely executing untrusted code (compile/run) and extract patterns for a multi-tenant task-code execution plane that reads/writes immutable spreadsheet artifacts with strict guardrails and auditability.

Strict evidence rule. All factual claims about Piston are backed by file paths and line ranges from the provided local checkout.

1) Repo & Runtime Overview

The API service is implemented in Node.js using Express; WebSocket support is enabled via express-ws. [api/src/index.js:L1-L17].

The API loads installed runtime packages from the configured data directory (default */piston*) under */piston/packages*, checking for the marker file *.ppman-installed* before loading a package. [api/src/config.js:L20-L26; api/src/index.js:L39-L62; api/src/globals.js:L75-L83].

The API does not implement isolation in-process. It shells out to an external sandbox binary *isolate* at */usr/local/bin/isolate* and uses per-execution isolate boxes. [api/src/job.js:L15-L17; api/src/job.js:L63-L88; api/src/job.js:L149-L180].

API endpoints (v2). The v2 router provides:

- *POST /api/v2/execute* (compile+run pipeline). [api/src/api/v2.js:L237-L265]
- *GET /api/v2/runtimes*. [api/src/api/v2.js:L267-L278]
- *GET/POST/DELETE /api/v2/packages*. [api/src/api/v2.js:L280-L320]
- *WS /api/v2/connect* (streaming). [api/src/api/v2.js:L136-L235]

Execution pipeline: request → result

Step	What happens	Evidence
Validate & select runtime	get_job() validates request fields, selects a semver-matching runtime, and enforces client limits ≤ configured maxima.	api/src/api/v2.js:L12-L119
Acquire concurrency slot	Job.prime() gates concurrency with an in-process semaphore + FIFO queue.	api/src/job.js:L19-L21; api/src/job.js:L90-L100
Create sandbox box	Job.#create_isolate_box() runs isolate --init --cg -b and records a box dir and a metadata file path in /tmp.	api/src/job.js:L63-L88
Materialize submission	prime() creates /submission and writes user files with a directory traversal escape check.	api/src/job.js:L102-L121
Compile (optional)	execute() calls safe_call(box,'compile',...) when runtime has a compile script.	api/src/job.js:L363-L385; api/src/runtime.js:L168-L174
Fresh sandbox for run	On successful compile, execute() creates a new isolate box and moves only the submission directory.	api/src/job.js:L377-L384
Run	execute() calls safe_call(box,'run',...) with stage-specific limits.	api/src/job.js:L387-L401
Return result	execute() returns {compile?, run?, language, version}. HTTP handler adds backward compat fallback when run is undefined.	api/src/job.js:L405-L410; api/src/api/v2.js:L247-L253
Cleanup & release slot	cleanup() runs isolate --cleanup --cg -b, removes metadata file, releases semaphore slot, and wakes queued jobs.	api/src/job.js:L413-L440

2) Formal Model: Sandbox as a Policy-Constrained Transition System

Model one execution as a transition system with explicit inputs, a sandbox policy Π, and a result/log record.

Inputs: execute request contains language/version/files/stdin/args plus optional per-stage overrides for memory/timeout/cpu time. [api/src/api/v2.js:L13-L25].

Policy Π: configured limits (defaults + overrides) plus the isolate invocation parameters used by safe_call(). [api/src/config.js:L51-L139; api/src/runtime.js:L39-L112; api/src/job.js:L149-L176].

Transition: safe_call() spawns isolate, collects or streams stdio, then reads isolate's metadata file to assemble cpu/wall/memory/status. [api/src/job.js:L149-L184; api/src/job.js:L200-L248; api/src/job.js:L264-L322].

Security invariants (must-holds) and enforcement points

Invariant	Property	Enforcement evidence
I1. Bounded resource usage	Per-stage bounds on wall time, CPU time, memory, process count, open files, per-file size.	safe_call passes --wall-time/--time/--extra-time=0, --cg-mem (if enabled), --processes, --open-files, --fsize. [api/src/job.js:L166-L174]. Limits come from config/overrides. [api/src/runtime.js:L39-L112; api/src/config.js:L63-L116].
I2. Output cannot exhaust host (HTTP path)	stdout/stderr capped and process aborted when exceeding output_max_size.	When event_bus is null, safe_call checks buffer length and SIGABRTs; sets status OL/EL. [api/src/job.js:L200-L248].
I3. Path-safe file injection	Client file names cannot escape submission root.	prime() rejects paths where path.relative(...) begins with '..'. [api/src/job.js:L105-L112].
I4. Default-deny networking	Outgoing networking disabled by default; enabled only by config.	disable_networking default true. [api/src/config.js:L51-L56]. --share-net only if disable_networking is false. [api/src/job.js:L175-L176].
I5. Bounded concurrency (single-process)	Total concurrent jobs limited per API process; excess requests queue.	remaining_job_spaces/job_queue gate prime(); cleanup releases slot. [api/src/job.js:L19-L21; api/src/job.js:L90-L96; api/src/job.js:L416-L419].
I6. Runtime package integrity (checksum)	Downloaded package archives verified via sha256 before extraction.	Package.install validates sha256(pkg.tar.gz) == checksum. [api/src/package.js:L69-L86].

Gap: In WebSocket streaming mode (event_bus != null), stdout/stderr size caps are bypassed (output is forwarded over WS). [api/src/job.js:L200-L208; api/src/job.js:L225-L233; api/src/api/v2.js:L140-L156].

3) Isolation Mechanisms

Primary mechanism: Isolate is invoked for box init and for execution. [api/src/job.js:L63-L88; api/src/job.js:L149-L176].

Cgroup v2 setup: entrypoint validates pure cgroup v2 and enables controllers under /sys/fs/cgroup before starting Node as user piston. [api/src/docker-entrypoint.sh:L3-L29].

Deployment note: docker-compose runs the API container privileged and mounts host packages into /piston/packages. [docker-compose.yaml:L4-L14].

README security claims: Isolate uses Linux namespaces, chroot, multiple unprivileged users, and cgroup. [readme.md:L458-L471].

Wrapper explicit controls: safe_call configures resource limits, directory access, and optional network sharing; it does not explicitly configure UID/GID in the isolate invocation shown. [api/src/job.js:L149-L180]. Config includes runner_uid/gid ranges but they are not referenced in api/src/job.js. [api/src/config.js:L27-L50; api/src/job.js:L149-L180].

4) Resource Limiting & Abuse Prevention

Per-stage limits: safe_call passes processes, open files, file size, wall time, CPU time, and optional cgroup memory to isolate. [api/src/job.js:L166-L175].

Default caps include disable_networking=true; output_max_size=1024 bytes; max_process_count=64; max_open_files=2048; max_file_size=10MB; compile_timeout=10s; run_timeout=3s; compile_cpu_time=10s; run_cpu_time=3s; compile/run_memory_limit=-1 (no limit). [api/src/config.js:L51-L116].

Admission control: get_job() rejects client-specified limits that exceed configured maxima. [api/src/api/v2.js:L70-L96].

Edge-case behaviors: if (!constraint_value) continue means 0 skips validation but is propagated via nullish coalescing. [api/src/api/v2.js:L70-L77; api/src/api/v2.js:L99-L116]. When configured_limit <= 0, the loop continues before checking < 0. [api/src/api/v2.js:L83-L95].

Concurrency: single-process semaphore with FIFO queue; max_concurrent_jobs default 64. [api/src/job.js:L19-L21; api/src/config.js:L123-L128].

5) Filesystem & Network Controls

Submission workspace: files written under <*box.dir*>*/submission*. Path traversal blocked via path.relative escape check; subdirs created with mode 0o700. [api/src/job.js:L102-L121].

Sandbox working directory and mounts: working dir /box/submission; runtime pkgdir allowed via --dir=; /etc mounted noexec. [api/src/job.js:L157-L165].

Network egress: default deny (disable_networking=true); enabling adds --share-net. [api/src/config.js:L51-L56; api/src/job.js:L175-L176].

6) Observability & Auditing

Logs: jobs have a UUID and a per-job logger name job/. [api/src/job.js:L35-L38].

Returned telemetry: safe_call parses isolate metadata and returns cpu_time, wall_time, memory, status, message, exit/signal plus stdout/stderr/output. [api/src/job.js:L264-L322].

Correlation/audit gaps: HTTP /execute returns compile/run plus language/version, but does not include job UUID/correlation ID. [api/src/job.js:L405-L410; api/src/api/v2.js:L237-L265]. On error, handler returns empty 500 body. [api/src/api/v2.js:L254-L256].

WS mismatch risk: WS emits event_bus 'signal' but safe_call listens for 'kill'. [api/src/api/v2.js:L210-L216; api/src/job.js:L190-L197].

7) Mapping to Our Platform

Mapping of our execution-plane needs to Piston mechanisms (with evidence), plus reuse/adaptation notes and risks.

Need	Piston mechanism	Evidence	Reuse/adaptation	Risks
Strong isolation boundary	Isolate per job box init + run with cgroups	api/src/job.js:L63-L88 ; L149-L176	Reuse external runner pattern; strengthen outer boundary and make identity explicit (UID/GID, mounts, egress).	Compose uses privileged container. [docker-compose.yaml:L4-L14]
Immutable spreadsheet artifacts	Materialize inputs into submission dir; runtime pkgdir allowed via --dir	api/src/job.js:L102-L121; L164-L165	Mount inputs read-only; stage outputs to dedicated writable dir; commit immutable versions with digests.	Writable inputs risk contamination; copying affects performance.
Resource limits	process/open-files/fsize/wall/cpu/mem passed to isolate	api/src/job.js:L166-L175; api/src/config.js:L51-L116	Represent limits as policy object; effective=min(requested, policy) with reason codes.	Validation edge-cases in get_job. [api/src/api/v2.js:L70-L96]
Controlled egress + child-task execution	Boolean network toggle via --share-net	api/src/config.js:L51-L56; api/src/job.js:L175-L176	Prefer declarative spawn intents; orchestrator enforces depth/spawn budget/cycle detection and tenant circuit breakers.	All-or-nothing networking too coarse; exfil risk.
Deterministic-ish run record	Returns stdio + isolate metadata stats	api/src/job.js:L264-L322	Add: code hash, runtime digest, input artifact digests, policy log, signed execution record.	No tamper-evidence/persistence in current code path.
Concurrency + fairness	In-process semaphore + FIFO queue	api/src/job.js:L19-L21 ; L90-L96; L416-L419	Replace with durable queue + per-tenant quotas/circuit breakers; persist admission decisions.	Not horizontally scalable; restart loses state.

8) Patterns to Steal vs Avoid (10+)

Patterns to steal

• **Thin wrapper around sandbox runner.** Delegates execution to isolate. [api/src/job.js:L149-L184].

• **Server-side limit admission control.** Reject constraints exceeding maxima. [api/src/api/v2.js:L70-L96].

• **Two-stage compile/run.** Separate stages with stage-specific limits. [api/src/job.js:L363-L401].

• **Fresh sandbox between compile/run.** New box created; only submission moved. [api/src/job.js:L377-L384].

• **Traversal-safe file materialization.** Block escape from submission root. [api/src/job.js:L105-L112].

• **Default-deny networking.** disable_networking default true. [api/src/config.js:L51-L56].

• **Multi-dimensional caps.** processes/open-files/fsize/time/memory enforced via isolate flags. [api/src/job.js:L166-L175].

• **Runner-generated telemetry.** Parse metadata file for cpu/wall/mem/status. [api/src/job.js:L264-L322].

• **Runtime checksum verification.** sha256 archive validated pre-extract. [api/src/package.js:L69-L86].

• **Concurrency cap.** max_concurrent_jobs semaphore. [api/src/job.js:L19-L21; api/src/config.js:L123-L128].

Patterns to avoid

• **Privileged container default.** docker-compose uses privileged: true. [docker-compose.yaml:L4-L14].

• **All-or-nothing networking.** Only knob is conditional --share-net. [api/src/job.js:L175-L176].

• **No authn/authz in router.** Only content-type middleware exists. [api/src/api/v2.js:L122-L134].

• **In-memory queue.** Process globals remaining_job_spaces/job_queue. [api/src/job.js:L19-L21].

• **WS output quota bypass.** Streaming mode bypasses size caps. [api/src/job.js:L200-L208; L225-L233; api/src/api/v2.js:L140-L156].

• **WS signal mismatch.** WS emits 'signal' but safe_call listens for 'kill'. [api/src/api/v2.js:L210-L216; api/src/job.js:L190-L197].

• **Tar extraction hardening not visible.** tar xzf via bash -c. [api/src/package.js:L92-L105].

• **Docs drift vs behavior.** Docs show /piston/jobs/ path; runner uses /box/submission. [docs/api-v2.md:L116-L126; api/src/job.js:L157-L165].

• **Opaque 500 responses.** POST /execute returns empty body on 500. [api/src/api/v2.js:L254-L256].

9) Actionable Output

Five ADRs for our execution plane

ADR-001 Sandbox boundary choice. Isolate + cgroup v2 setup. [api/src/job.js:L149-L176; api/src/docker-entrypoint.sh:L3-L29]. Compose runs privileged. [docker-compose.yaml:L4-L14]. Decision: stronger outer boundary and explicit identity.

ADR-002 Limits as a first-class policy object. Per-language limits (defaults + overrides) and constraint checks. [api/src/runtime.js:L39-L112; api/src/api/v2.js:L70-L96]. Decision: centralize limits; record effective limits + reason codes.

ADR-003 Egress policy and child-task spawning. Boolean network toggle only. [api/src/config.js:L51-L56; api/src/job.js:L175-L176]. Decision: deny arbitrary network in sandbox; use declarative spawn intents or mediated internal channel.

ADR-004 Artifact mount strategy. Submission materialization with traversal checks. [api/src/job.js:L102-L121]. Decision: mount spreadsheet artifacts read-only; stage outputs; commit immutable versions after validation.

ADR-005 Audit-grade records and tamper-evidence. Telemetry available from isolate metadata but no signed audit pipeline. [api/src/job.js:L264-L322; api/src/api/v2.js:L237-L265]. Decision: emit signed ExecutionRecords binding code hash + runtime digest + artifact digests + policy version + outputs.

Reference Execution Contract (draft)

```
Inputs
- execution_id (uuid), idempotency_key, tenant_id, task_id/task_version, partition_key
- code: language, runtime_selector, entrypoint, files[], code_sha256
- artifacts:
  - inputs[]: {artifact_name, artifact_version, digest, mount_path, mode=ro}
  - outputs[]: {artifact_name, staging_path, mode=rw}
- limits: wall_time_ms, cpu_time_ms, memory_bytes, max_processes, max_open_files, max_output_bytes
- policy: policy_version, network policy (deny-by-default + allowlist), spawn_guardrails (max_depth
          tenant circuit breaker (max concurrency/failures)
- env: explicit allowlisted env vars

Outputs
- status: succeeded|failed|canceled|rejected
- exit: code/signal, timing: wall/cpu, resources: peak memory
- logs: stdout/stderr (bounded or external refs)
- outputs[]: new artifact versions + digests
- spawn: requested/admitted/denied with reason codes
- policy_decisions[]: allow/deny, effective limits, reasons, policy_version
- provenance: runtime_digest, input digests, execution_record_hash, signature
```